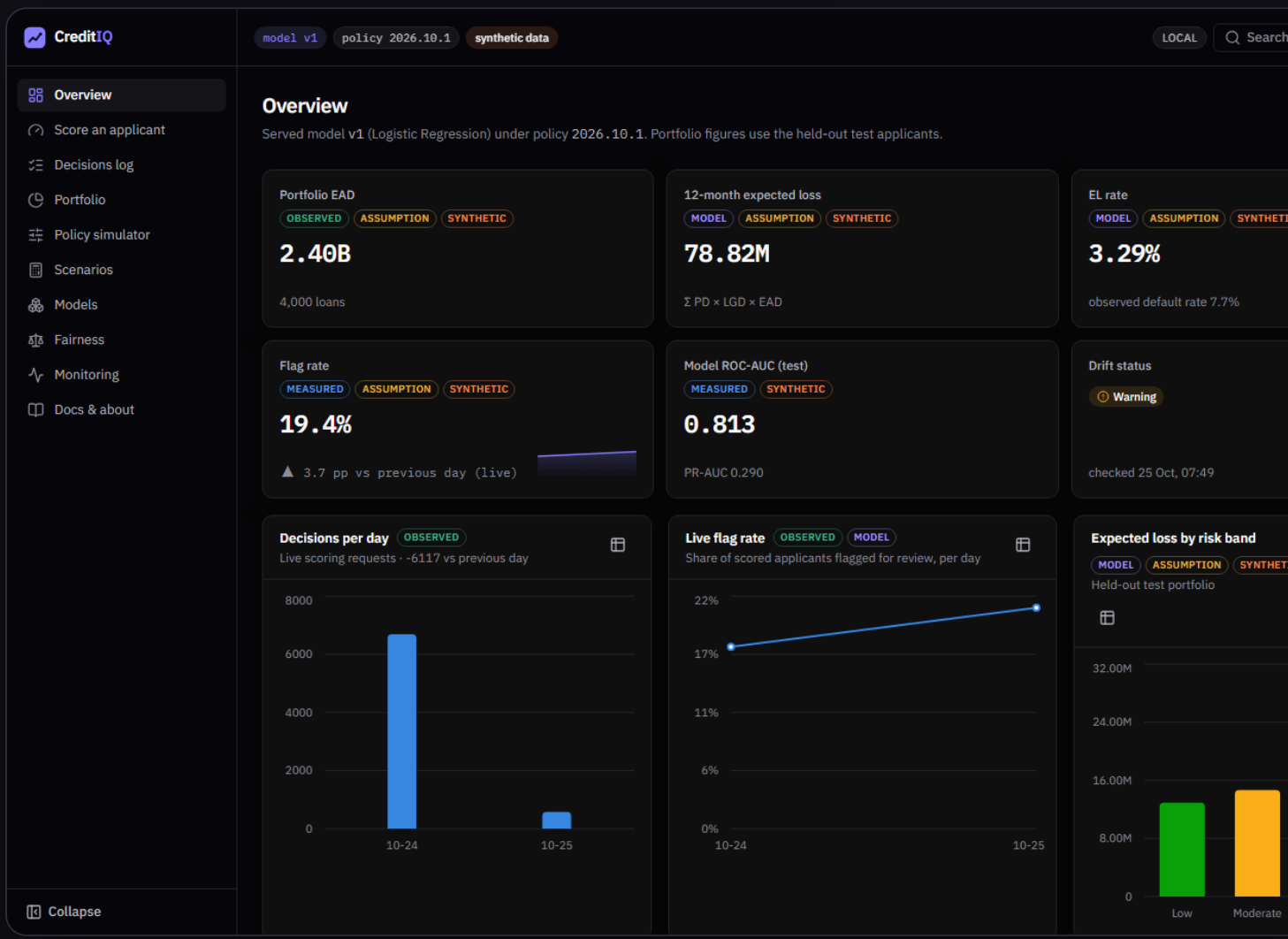
CreditIQ

01 The app in one minute
02 How it is built
03 The two workflows
04 Key ideas behind the numbers
05 Feature tour
06 How to run it
07 Five-minute demo
08 Troubleshooting & glossary

Team guide to CreditIQ v2

How the app works, what every feature does, and how to run it on your own laptop.

Python · FastAPI | scikit-learn · LightGBM | Next.js · TypeScript | MLflow



Group project · October 2026
github.com/Bikram-Dev-Web/credit-iq

Educational decision-support prototype, not for real credit decisions.

01 · OVERVIEW

What CreditIQ does

CreditIQ estimates how likely a loan applicant is to run into repayment trouble, checks the result against a lending policy, explains why, and keeps an audit trail. It is a decision-*support* tool: a person makes the decision.

1 · Predict the risk

A machine-learning model reads the application (income, loan amount, external credit scores…) and outputs a **probability of default (PD)**, for example 6.0%.

2 · Apply the policy

A separate, versioned **lending policy** compares the PD with a threshold and runs rules (e.g. instalment above 45% of income). Result: *within threshold* or *flagged for review*.

3 · Explain, audit, watch

Every answer lists its top reasons, is written to an **audit log** (without personal data), and a **monitor** warns when incoming applicants start to look different from the training data.

The numbers worth remembering

0.748
test ROC-AUC on the real Home Credit data (LightGBM)

0.813
test ROC-AUC on the bundled synthetic data (logistic regression)

10
dashboard pages, 35 API endpoints

178
automated Python tests + browser tests, run by CI on every push

Two datasets

Synthetic sample SYNTHETIC

20,000 fabricated applications shipped in the repo (`data/sample_credit_data.csv`). Used by default, so the app runs for anyone straight after cloning. Its results say nothing about real borrowers.

Home Credit (real) OBSERVED

The free Kaggle *Home Credit Default Risk* file. Not in the repo (too large, licence). Download `application_train.csv` into `data/raw/` to train the real model (page 8).

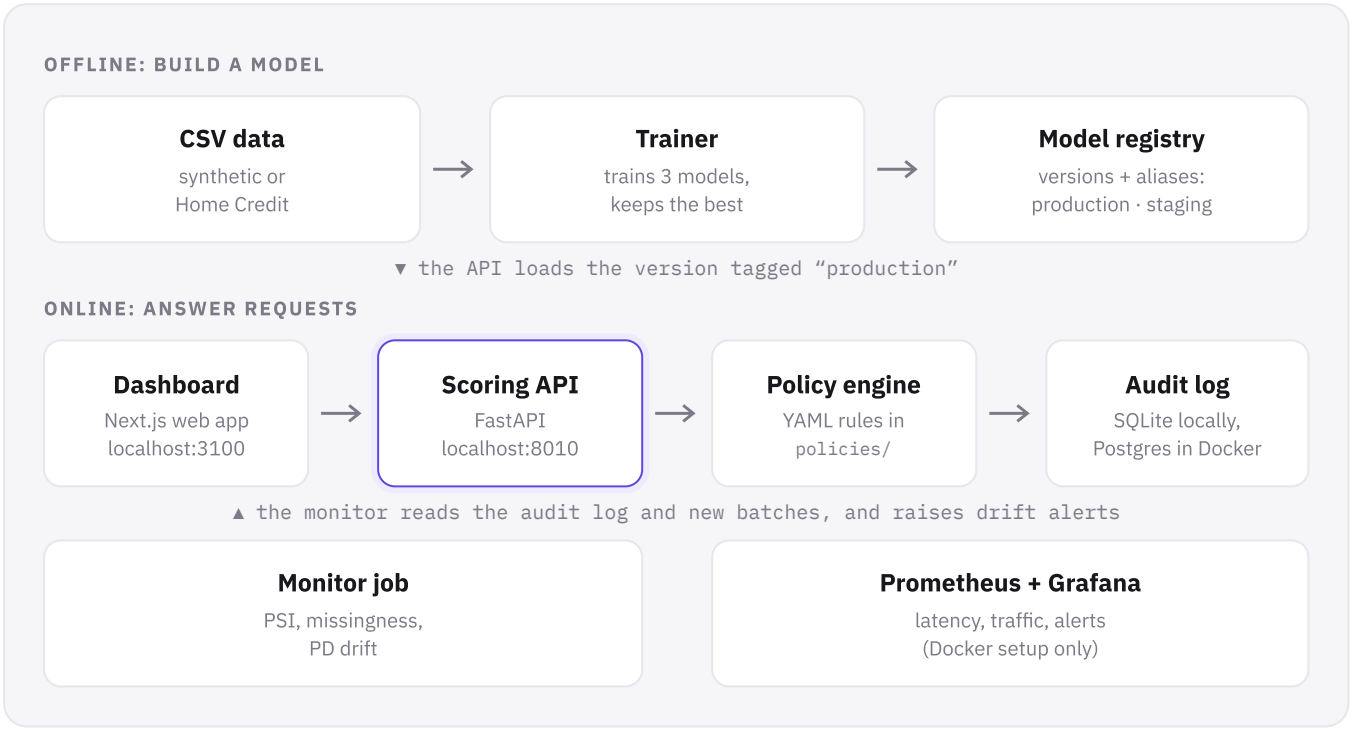
What it deliberately does not do

Never approve or deny. Outcomes are only “within threshold” or “flagged for review”. Gender, age and family status are never model inputs; they are used only to *check* fairness. The applicant’s raw inputs are never stored. This is a university prototype, not a tool for real lending.

02 · ARCHITECTURE

Five pieces, one shared library

All the maths lives in one Python library, `creditiq_core`. The trainer, the API and the monitor import it, so a number is computed the same way everywhere.



Who does what

FOLDER	ROLE
packages/creditiq_core	The library: data checks, features, models, calibration, explanations, fairness, drift, lending economics, policy engine.
services/trainer	`creditiq-train`: trains from a YAML config, writes a versioned model package, promotes and rolls back.
services/api	`creditiq-api`: the REST API under `/v1` (score, decide, models, policies, portfolio, fairness, monitoring…).
services/monitor	`creditiq-monitor`: scheduled drift checks over the audit log and an inbox of CSV batches.
web	The dashboard. It never holds API keys in the browser: a small server-side proxy adds them.
policies	Lending policies as versioned YAML files (`2026.10.1` baseline, `2026.10.2-strict`).
legacy	The original v1 Streamlit app, unchanged. v2 reproduces its numbers exactly.

03 · WORKFLOW

From data to a decision

There are two loops. Training happens rarely and offline; scoring happens on every request, in tens of milliseconds.

A · Training a model

1. **Load and check the data**
   Schema and quality checks; the target mapping must be written in the config (which value means "default").
   `configs/*.yaml`
2. **Split 60 / 20 / 20**
   Train, validation and test, stratified, seed 42. Test is touched once, at the very end.
3. **Build features**
   Ratios such as credit ÷ income and instalment ÷ income; fill gaps, encode categories. Sensitive attributes are removed.
4. **Train three model families**
   Logistic regression, random forest, LightGBM. The best **validation** ROC-AUC wins.
5. **Calibrate and pick the threshold**
   Turn raw scores into honest probabilities and choose the review threshold, both on validation only.
6. **Evaluate on test, register**
   Final metrics, model card and quality report go into a new registry version, tagged *staging* or *production*.
   `models/registry/creditiq-pd`

B · Life of one decision

1. **Analyst fills the form**
   On *Score an applicant*. The form is generated from the model's own input list.
2. **Proxy adds the key**
   The web server forwards the request to the API with the API key, so the key never reaches the browser.
   `POST /v1/decide`
3. **Validate and score**
   Bad fields get a clear error. The model returns a calibrated PD and the top reasons.
4. **Apply the policy**
   PD vs threshold, plus hard rules. Outcome: within threshold or flagged for review, with reasons.
5. **Write the audit record**
   Model and policy versions, scores, rules fired, and a keyed hash of the inputs. If the database is down, no answer is given (503).
6. **Show the result**
   PD gauge, risk band, expected loss, the reasons chart. The monitor later uses these scores to spot drift.

Swapping models without downtime. When an admin promotes another version, the API notices within 30 seconds, loads it in the background and switches over in one step. Requests in flight finish on the old model.

04 · CONCEPTS

The ideas behind the numbers

Where each number on screen comes from.

Probability of default (PD)

The model's estimate that this applicant has the dataset's "default" event. For Home Credit that event is **early payment difficulty**, not a 12-month default. A PD of 6% means: out of 100 similar applicants, about 6 would have trouble.

The review threshold (e.g. 14.4%)

Chosen automatically at training time: the trainer tries every cut-off on the **validation** set and keeps the one with the best F1 score (balance of catching defaults vs flagging good applicants). Synthetic model: **14.4%**. Home Credit model: **12.3%**. The policy can override it per segment (revolving loans: 12%).

Risk bands

Low	Moderate	Elevated	High
PD < 5%	5 – 10%	10 – 20%	≥ 20%

Bands are set by the policy (`band_cutoffs: [0.05, 0.10, 0.20]`). Bands describe risk; only the threshold and the rules decide the outcome.

Expected loss

`EL = PD × LGD × EAD`

LGD (loss given default) is an *assumption* of 45%; EAD is the loan amount. In the screenshot on page 6: 6.0% × 45% × 487,500 ≈ 13,060.

"Why this score"

Each input's exact contribution, in log-odds. Red bars push the risk up, blue bars push it down. It shows **association, not causation**.

Provenance tags: every number says where it came from

TAG	MEANING	EXAMPLE
OBSERVED	Counted from real records	Decisions per day, portfolio size
MEASURED	Measured on held-out data	ROC-AUC 0.813, flag rate
MODEL	Output of the model	PD 6.0%, raw score
ASSUMPTION	A number we chose	LGD 45%, interest rate 16%
SIMULATED	Result of a what-if or stress test	Policy simulator, scenarios
SYNTHETIC	Based on fabricated data	Anything from the bundled sample

Why results are trustworthy. Calibration and the threshold use validation only, the test split is used once, leakage checks run on training rows, and sensitive attributes stay out of the model. These rules are covered by automated tests.

05 · FEATURE TOUR

Every page in the dashboard

Ten pages in the left sidebar. The top bar always shows the model version, the policy version and whether the data is synthetic.

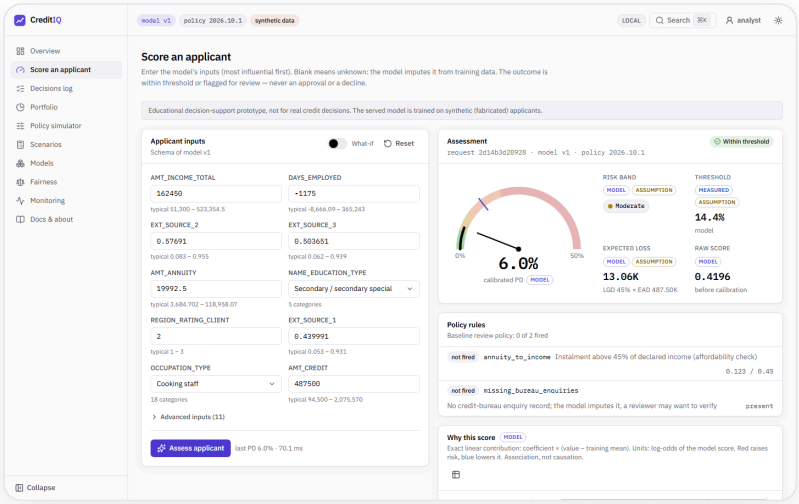
01 Overview

The home page: portfolio size, 12-month expected loss, flag rate, model ROC-AUC, drift status, and charts of daily decisions and loss by risk band.

02 Score an applicant

The main feature. Enter an application and press **Assess applicant**. You get the PD gauge, risk band, expected loss, threshold, which policy rules fired, and the reasons chart.

Turn on **What-if** and the result updates live as you change inputs. Blank fields are fine: the model fills them from training data.

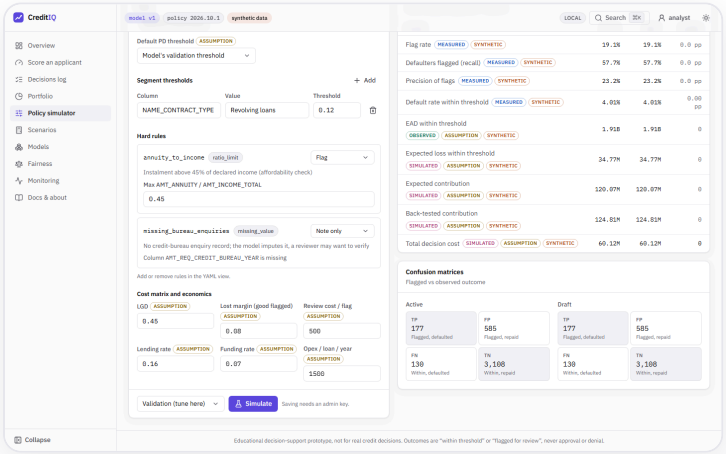


03 Decisions log

The audit trail: every decision with time, model and policy versions, PD, outcome, rules fired and reasons. Filter and open any row.

04 Portfolio

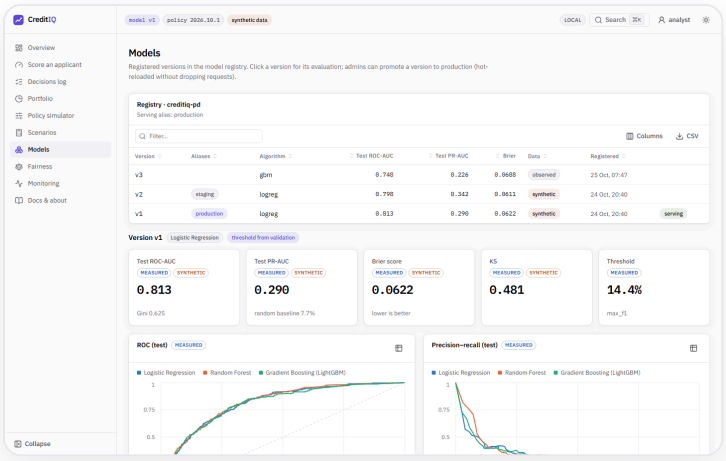
The held-out test applicants as a loan book: exposure and expected loss by band, predicted vs observed default rate by decile, concentration (HHI) and loss curves.



05 Policy simulator

Change thresholds, rules and costs in a form or as YAML, then **simulate**: flag rate, expected loss, profit and the confusion matrix, side by side with the active policy.

An `ADMIN` can save the draft as a new policy version and activate it. No retraining is needed.



06 Models

Every registered version with its metrics, ROC and precision-recall curves, calibration chart, model card and a threshold explorer (validation data only).

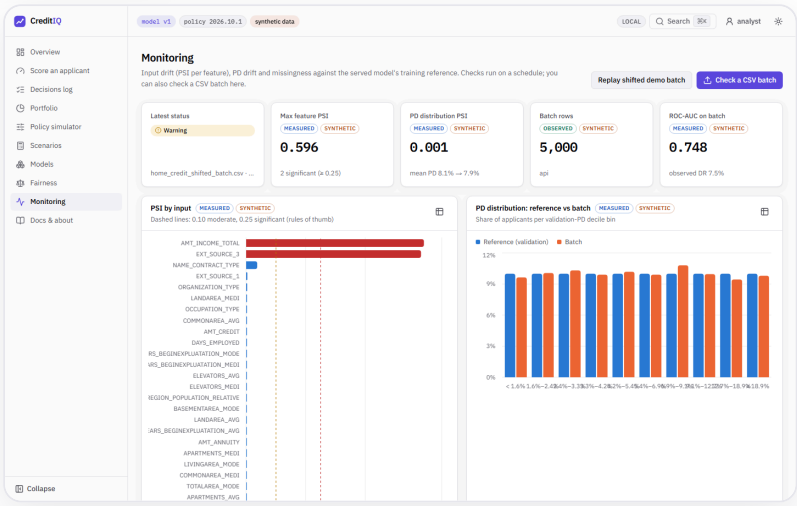
An `ADMIN` can **promote** a version to production or **roll back** to the previous one. The API switches within 30 seconds.

07 Monitoring

Is today's data still like the training data? Shows drift per input (PSI), PD drift, missing values, and the alert history.

Replay shifted demo batch runs a drift check on a prepared batch that matches the live model, and should raise a **warning**. **Check a CSV batch** lets you upload your own file.

PSI below 0.10 is fine, 0.10–0.25 is a warning, above 0.25 is significant.



08 Scenarios

Lending economics: break-even PD and interest rate, a PD × rate profit heatmap, stress tests (higher PDs or losses) and loan NPV.

09 Fairness

Compares flag rates and errors across gender, age and family-status groups, plus a scan for inputs that act as stand-ins for them.

10 Docs & about

Method, limitations, the disclaimer and links to the API docs, MLflow and Grafana when they are running.

Two roles

Analyst `DEFAULT`

Can score, browse, simulate and run drift checks. Cannot change what is live.

Admin `SWITCH IN THE TOP BAR`

Can also promote or roll back models and save or activate policies. Every admin action is logged.

06 · SETUP

Run it on your laptop

No cloud account and no Docker needed. You need three free tools, then one command.

Git

git-scm.com, to clone the repository.

uv

The Python tool. It installs Python 3.12 and every package for you.

`astral.sh/uv`

Node.js 20+ and pnpm

For the dashboard. After installing Node, run `corepack enable`.

Option A · one command (Windows)

```
# 1. get the code and install the Python side
git clone https://github.com/Bikram-Dev-Web/credit-iq.git
cd credit-iq
uv sync

# 2. start everything (first run trains the model and builds the web app: ~5 minutes)
powershell -ExecutionPolicy Bypass -File scripts\run-local.ps1
```

Two minimized PowerShell windows open: one for the API and one for the dashboard. Close them to stop the app.

Option B · step by step (macOS, Linux or Windows)

```
uv sync
uv run creditiq-train train --config services/trainer/configs/synthetic.yaml \
    --no-mlflow --file-registry models/registry --alias production
# terminal 1: the API
API_KEYS="analyst:dev-analyst-key-change-me,admin:dev-admin-key-change-me" uv run creditiq-api
# terminal 2: the dashboard
cd web && pnpm install && cp .env.example .env.local && pnpm dev
# optional: fill the decisions log with demo traffic
uv run python scripts/seed_demo.py --key dev-admin-key-change-me
```

Where to open it

LINK	WHAT
localhost:3100	Dashboard (option A)
localhost:8010/docs	Interactive API docs
localhost:3000 / 8000	Same, with option B

API key for the docs page: `dev-analyst-key-change-me` (development only).

Use the real Home Credit model

- Download `application_train.csv` from Kaggle (free account) into `data/raw/`.
- Train: `uv run creditiq-train train --config services/trainer/configs/home_credit.yaml --no-mlflow --file-registry models/registry --alias production`
- Restart the API. It needs **2–3 minutes** to load the bigger model.

07 · PRESENTING

A five-minute live demo

Start the app at least five minutes before you present, and open each page once so it is warm.

1. **Overview** — 0:00 – 0:45

   “One model, one policy, every number tagged with where it comes from.” Point at the version chips in the top bar.

2. **Score an applicant** — 0:45 – 2:00

   Press **Assess applicant**. Explain the PD, the band, the threshold and the reasons. Turn on **What-if** and lower `EXT_SOURCE_2` to watch the PD rise until it is flagged.

3. **Policy simulator** — 2:00 – 3:00

   Lower the threshold, press **Simulate**: more applicants flagged, fewer missed defaults, more review work. That is the trade-off a lender makes.

4. **Models** — 3:00 – 3:45

   Show the model card and calibration. As admin, promote another version and say: “the API switches without downtime.”

5. **Monitoring** — 3:45 – 4:30

   Press **Replay shifted demo batch**; a warning appears with the inputs that changed.

6. **Decisions log** — 4:30 – 5:00

   Every decision we just made is here, with versions and reasons, and without the applicant’s data.

Questions you will probably get

Why not just approve or deny?

A model is a tool for a reviewer, not a judge. “Flagged for review” keeps a person accountable.

Is 0.748 good?

It is in the usual range for this dataset when only the main application table is used. Adding the bureau and previous-loan tables is the usual way to go higher.

Is it fair?

Sensitive attributes are excluded and fairness is measured per group; small groups give noisy numbers, and we say so.

How fast is it?

Synthetic model: p95 46 ms at 20 requests/s. The real model: p95 73 ms, still above our 50 ms target.

08 · REFERENCE

When you get stuck

If something goes wrong

PROBLEM	FIX
Dashboard says the API is unreachable	The API is still loading (up to 3 min for the real model). Check `localhost:8010/healthz`.
“Port already in use”	Another app uses the port. Option A uses 8010/3100 for that reason; close the other app or change `PORT`.
Script blocked by Windows	Keep `-ExecutionPolicy Bypass` in the command.
Dashboard looks outdated after a pull	Delete `web/.next` and run the script again to rebuild.

Glossary

PD · probability of default
The model’s estimate that an applicant will have the default event.

LGD · loss given default
Share of the loan lost if a default happens. Assumed 45%.

EAD · exposure at default
How much is owed when the default happens; here, the loan amount.

ROC-AUC
How well the model ranks risky above safe applicants: 0.5 is a coin flip, 1.0 is perfect.

Calibration
Adjusting scores so a 10% PD really means about 10 in 100 have trouble.

F1 score
A balance of catching defaults (recall) and not flagging good applicants (precision). Used to pick the threshold.

PSI · population stability index
How much an input’s distribution has moved since training. Above 0.10 is a warning.

Registry alias
A label such as *production* pointing at one model version. Moving it is a promotion.

Where to read more in the repo

`README.md`	Overview, results, quick start
`docs/MODEL_CARD.md`	What the model is for, its data, metrics and limits
`docs/RUNBOOK.md`	Operating it: promote, roll back, handle alerts
`docs/adr/`	15 short records of why each technical choice was made